Context & Handlebars, Explained

One example workflow: Manual Trigger → HTTP Request (GET) → Gemini AI → Telegram

Nodebase • Every executor forks the context object • Handlebars compiles exactly what you wrote, at execution time • Generated 2026

context.myApiCall | context.myGemini | {{json ...}} helper | compile() then render(context) | Topological order

1. The Example Workflow

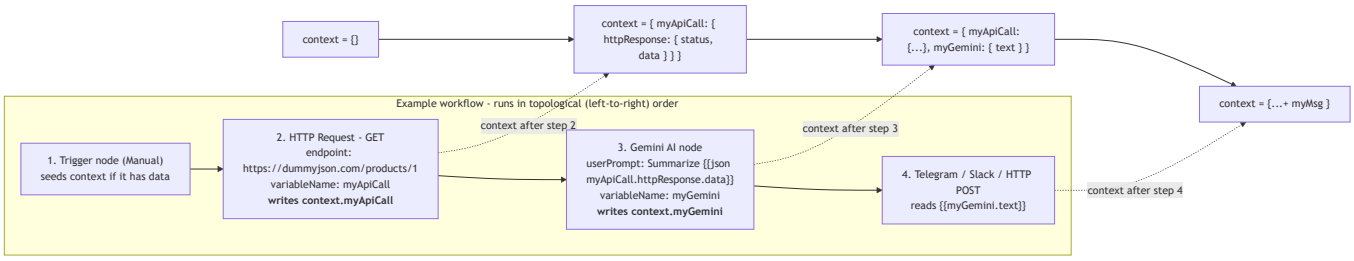


Figure 1 — Each node adds one key to the shared context bag; the next node sees everything added before it

The idea to hold on to: **a workflow is a single JavaScript object called `context` that is passed from node to node**. Every node:

- receives the current `context` (everything previous nodes produced),
- reads whatever it needs from it using Handlebars `{{...}}` expressions,
- returns a *new* context that keeps all old keys and adds its own output under the *variable name* you typed in its dialog.

```
// src/inngest/functions.ts - the loop that threads context through the graph
let context = event.data.initialData || {};      // starts as {} (or trigger payload)

for (const node of sortedNodes) {
  context = await executor({
    data: (node.data as any) || {},   // what YOU configured in the node dialog
    nodeId: node.id,
    userId,
    context,                          // everything from the nodes before this one
    step,
    publish,
  });
}
```

What you will type in each node's dialog

Node	Field	Value you type
1. Trigger	Manual trigger	nothing special
2. HTTP Request	Variable Name	`myApiCall` — the key its result will live under
	Endpoint URL	`https://dummyjson.com/products/1`
	Method	`GET`
3. Gemini AI	Variable Name	`myGemini`
	System Prompt (optional)	`You are a concise product analyst.`

	User Prompt	`Here is the data: {{json myApiCall.httpResponse.data}}. Write one line about {{myApiCall.httpResponse.data.title}} costing ${{myApiCall.httpResponse.data.price}}.`
4. Telegram	Chat ID	`(your chat id)`
	Text	`Summary: {{myGemini.text}}`

2. What Handlebars Actually Does

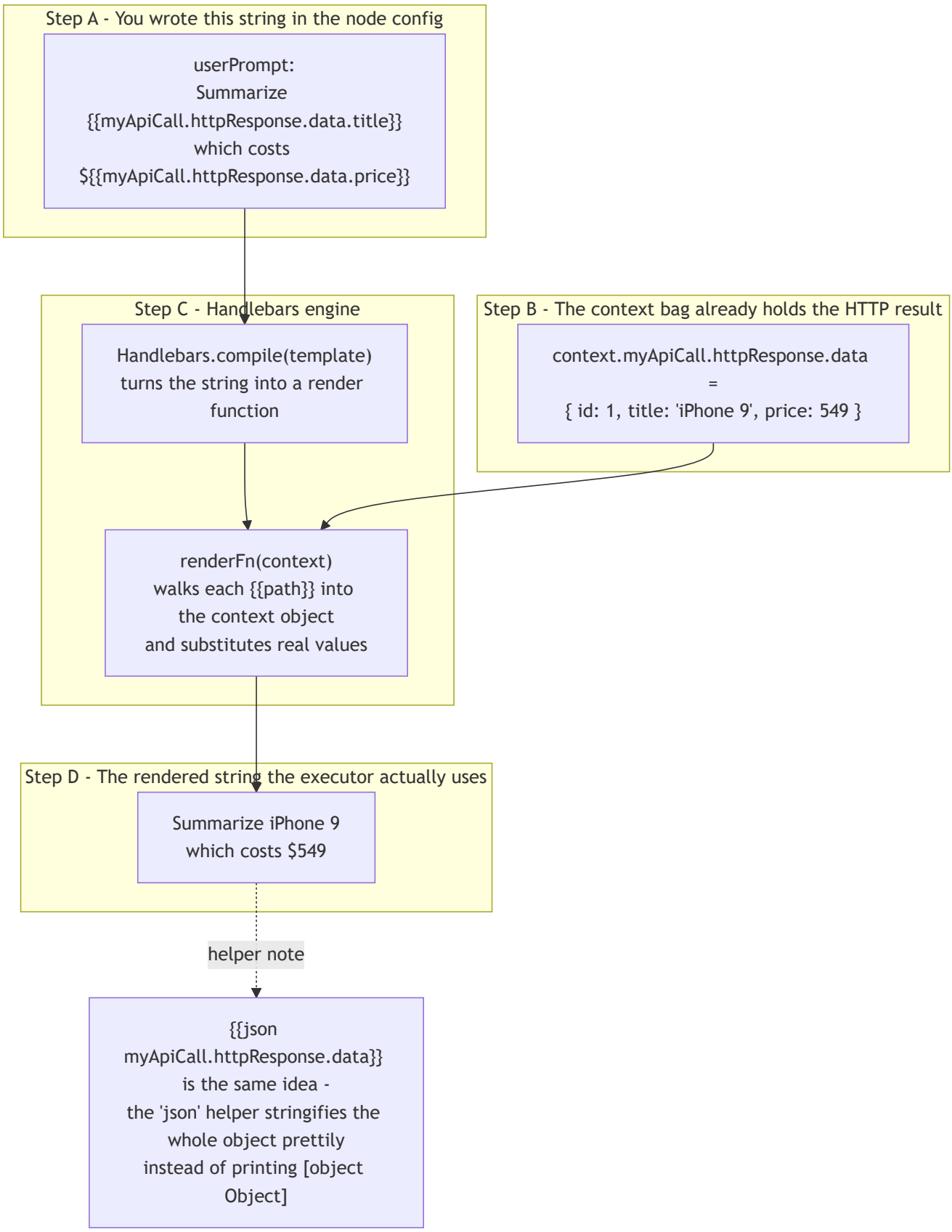


Figure 2 — compile(template) → a function; renderFn(context) → the real string. The executor never sends a template to Gemini -- only the rendered string

The rules, plainly

Rule	Explanation
`{{a.b.c}}` = "walk the object"	Handlebars reads `context.a.b.c`. If `a` or `a.b` doesn't exist, it renders an **empty string** (no error).
Two phases	`Handlebars.compile(str)` just builds the render function; calling it *with context* substitutes the values. That's why executors do it at runtime — the data isn't known until the run happens.
`{{json someObject}}`	A registered helper that does `JSON.stringify(x, null, 2)` and returns it safe. Needed for objects/arrays — writing `{{myApiCall.httpResponse.data}}` alone would print `[object Object]`.
Everything else is literal	Plain text (including `$`, quotes, newlines) is passed through verbatim. Only `{{...}}` is special.
Compiled at runtime	Because the template is stored as a string in `node.data`, the executor compiles & renders it fresh on every run with the then-current context.

3. Step-by-Step (What Actually Happens)

Step 1 — HTTP Request runs and writes context.myApiCall

When the HTTP node executes, the executor does (http-request/executor.ts):

```
const endpoint = Handlebars.compile(data.endpoint)(context);
// your value "https://dummyjson.com/products/1" has no {{...}},
// so it comes back unchanged.

const response = await ky(endpoint, options);        // GET https://dummyjson.com/products/1
const responseData = await response.json();          // { id: 1, title: "iPhone 9", ... }

return {
  ...context,                                  // keep everything from before
  [data.variableName]: {                       // "myApiCall"
    httpResponse: { status: response.status, statusText: response.statusText, data: responseData },
  },
};
```

Context right after the HTTP node (this is the object Gemini will receive):

```
{
  "myApiCall": {
    "httpResponse": {
      "status": 200,
      "statusText": "OK",
      "data": {
        "id": 1,
        "title": "iPhone 9",
        "description": "An apple mobile which is nothing like apple",
        "price": 549,
        "rating": 4.69,
        "stock": 94,
        "brand": "Apple",
        "category": "smartphones"
      }
    }
  }
}
```

Step 2 — Gemini runs and turns your prompt into a real string

The Gemini executor compiles your two prompts against the context from Step 1 (gemini/executor.ts):

```
const systemPrompt = Handlebars.compile(data.systemPrompt)(context);
const userPrompt   = Handlebars.compile(data.userPrompt)(context);
const { steps } = await step.ai.wrap("gemini-generate-text", generateText, {
  model:  google("gemini-flash-latest"),
  system: systemPrompt,
  prompt: userPrompt,
});
```

Your template (what was saved in `node.data.userPrompt`):

```
Here is the data: {{json myApiCall.httpResponse.data}}.
Write one line about {{myApiCall.httpResponse.data.title}} costing ${{myApiCall.httpResponse.data.price}}.
```

What gets sent to Gemini (the rendered string):

```
Here is the data: {
  "id": 1,
  "title": "iPhone 9",
  "description": "An apple mobile which is nothing like apple",
  "price": 549,
  "rating": 4.69,
  "stock": 94,
  "brand": "Apple",
```

```
  "category": "smartphones"
}.
Write one line about iPhone 9 costing $549.
```

Notice: `{{json myApiCall.httpResponse.data}}` became the whole object (pretty-printed), `{{myApiCall.httpResponse.data.title}}` became `iPhone 9`, and `${{myApiCall.httpResponse.data.price}}` became `$549` — the `$` is just literal text next to the placeholder.

The Gemini node then returns (gemini/executor.ts):

```
return { ...context, [data.variableName]: { text } };
//  context.myGemini.text = "iPhone 9 is a 549 USD smartphone by Apple with a 4.69 rating."
```

Context after the Gemini node:

```
{
  "myApiCall": { "httpResponse": { "status": 200, ... } },   // untouched, still there
  "myGemini":  { "text": "iPhone 9 is a 549 USD smartphone by Apple with a 4.69 rating." }
}
```

Step 3 — Telegram reads it with {{myGemini.text}}

The Telegram executor compiles `data.text` the same way (telegram-send-message/executor.ts):

```
Summary: {{myGemini.text}}
```

```
Summary: iPhone 9 is a 549 USD smartphone by Apple with a 4.69 rating.
```

That message string is what actually gets sent to Telegram. Same pattern any action node follows — HTTP Request bodies, Slack messages, Sheets rows: every field that supports `{{...}}` is compiled against this same growing context.

4. The Mental Model

Think of context as a backpack

- Nodes only walk left-to-right (topological order of your edges).
- Each node puts *one item in* (its variable name) and can *look at every item already inside*.
- A node can never see data from a node that runs after it.
- Same variable name used twice = the later node replaces the earlier key.

Think of Handlebars as find-and-replace

- `{{myApiCall.httpResponse.data.title}}` = look up that exact path, paste the value.
- `{{json <path>}}` = paste the whole thing as pretty JSON (for objects/arrays).
- Missing path → silently empty string (a sneaky source of "why is my prompt empty").
- Every `{{...}}` is resolved before the node runs its real work (HTTP call, AI request, message send).

Common mistakes & gotchas

What people type	What goes wrong	Fix
`{{myApiCall.httpResponse.data}}` (an object)	Renders `[object Object]`	`{{json myApiCall.httpResponse.data}}`
`{{apiCall...}}` (wrong variable name)	Empty string — no error	Use exactly the HTTP node's Variable Name (`myApiCall`)
Referencing a later node's output	Empty string, node ran before it existed	Reorder the edges so data producers come first
`data.length` on arrays	Works only because the whole object is in context — use `{{json ...}}` for arrays of objects	`{{myApiCall.httpResponse.data.products.length}}` if the API returns `{ products: [...] }`
Forgetting a credential/variable name on the node	`NonRetriableError` — run fails immediately (no retry)	Configure every required field

One-line summary

context is the shared object that moves through the workflow carrying every node's output (keyed by your variable names); **Handlebars** turns the *template string you wrote* into a real string by looking up `{{paths}}` inside that object, right before the node does its actual work.